

Reinforcement Learning: Value Function Approximation (ESE 3990)

Instructor:
Antonio Loquercio

Fall 2026
University of Pennsylvania

Today

- Solve a *known* MDP
 - Planning by Dynamic Programming
- Evaluate the value function of an *unknown* MDP
 - Model-free prediction
- Optimize the value function/policy of an *unknown* MDP
 - Model-free control
- Parametric approximation of value functions for complex, high-dimensional problems.

Large-Scale Reinforcement Learning

- Reinforcement Learning can be used to solve large problems, e.g.
 - Backgammon: 10^{20} states
 - Computer Go: 10^{170} states
 - Robot Control: continuous state space
- As a reference, the Universe has $\approx 10^{80}$ atoms.
- How can we scale the model-free methods for *prediction* and *control* from the last two lectures?

Value Function Approximation

- So far, we have represented value functions by *lookup tables*.
 - Every discrete state s has an entry $V(s)$
 - Or every state-action pair s, a has an entry $Q(s, a)$
- Problem with large MDPs:
 - There are too many states and/or actions to store in memory
 - It is too slow to learn the value of each state individually
 - Bellman's curse of dimensionality

Value Function Approximation

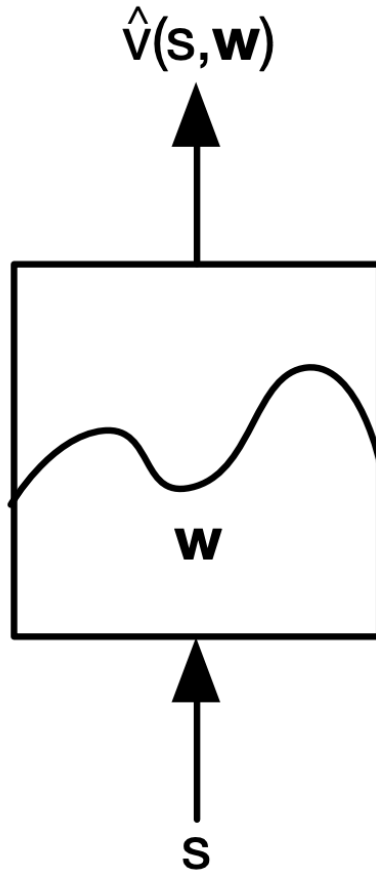
- **Idea.** For large MDPs, estimate the value function with *function approximation*.

$$\hat{V}(s, \mathbf{w}) \approx V^\pi(s)$$

$$\hat{Q}(s, a, \mathbf{w}) \approx Q^\pi(s)$$

- **Goal.** Generalize from seen states to unseen states
 - You can't access all possible states. So you *hope* that it works on unseen ones.
- **Implementation.** Update parameter $\mathbf{w}$ using MC or TD Learning.

Types of Value Function Approximation



Methods for Function Approximation

- There are many function approximators we can use:
 - Linear combination of features
 - Neural Networks
 - Decision trees
 - Nearest Neighbor
 - Fourier / Wavelet basis
 - ...

Differentiable Function Approximators

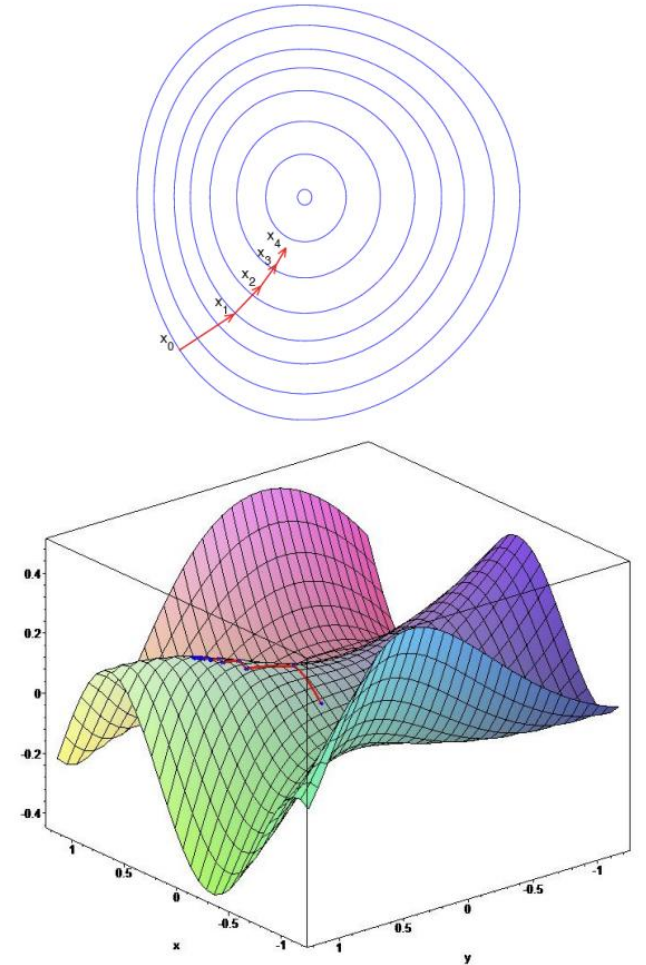
- We focus on **differentiable** function approximators:
 - **Linear combination of features**
 - **Neural Networks**
 - Decision trees
 - Nearest Neighbor
 - Fourier / Wavelet basis
 - ...

Learning Methodologies

- *Online (Incremental)*
 - We use samples online to update the value function
 - And discard them after
- *Batch*
 - We collect many samples and then use them together to update the value fn.
 - We keep a replay buffer of old samples.

Gradient Descent Refresher

- Let $J(\mathbf{w})$ be a differentiable function with parameter vector $\mathbf{w}$
- The gradient of $J(\mathbf{w})$ is defined as:
- $$\nabla_{\mathbf{w}} J(\mathbf{w}) = \begin{pmatrix} \frac{\partial J(\mathbf{w})}{\partial w_1} \\ \vdots \\ \frac{\partial J(\mathbf{w})}{\partial w_n} \end{pmatrix}$$
- We use the gradient to find a local minimum of $J(\mathbf{w})$
- By adjusting w in the direction of negative gradient:
 - $\Delta \mathbf{w} = -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w})$
 - Where α is the scalar step size.



Value Function Approx. By Stochastic Gradient Descent

- **Goal.** Find the parameter vector w minimizing mean-squared error between approximate value fn $\hat{V}(s, w)$ and the true value fn $V^\pi(s)$:

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[\left(V^\pi(s) - \hat{V}(s, \mathbf{w}) \right)^2 \right]$$

- **Gradient descent** finds a local minimum

$$\Delta \mathbf{w} = -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w}) = \alpha \mathbb{E}_\pi \left[\left(V_\pi(s) - \hat{V}(s, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s, \mathbf{w}) \right]$$

- **Stochastic gradient descent** *samples* the gradient

$$\Delta \mathbf{w} = \alpha \left(V_\pi(s) - \hat{V}(s, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s, \mathbf{w})$$

- In expectation, the two methods are equivalent (i.e., give same update)

Feature Vectors

- We represent the state s by a *feature vector*

$$\mathbf{x}(s) = \begin{pmatrix} x_1(s) \\ \vdots \\ x_n(s) \end{pmatrix}$$

- The feature vectors can be either the raw observation o_t (e.g., camera pixels) or a pre-processed version of it, such as:
 - Distance of the robot from landmarks and obstacles
 - Trends in the stock market

Linear Value Function Approximation

- Represent the value function by a linear combination of features

$$\hat{V}(s, \mathbf{w}) = \mathbf{x}(s)^T \mathbf{w} = \sum_{j=1}^n x_j(s) w_j$$

- The objective function is quadratic in parameters $\mathbf{w}$

$$J(\mathbf{w}) = \mathbb{E}_{\pi}[(V^{\pi}(s) - \mathbf{x}(s)^T \mathbf{w})^2]$$

- Convex function wrt w , so stochastic gradient descent converges on the *global* optimum.

- Does this mean we have a perfect approximation of $V^{\pi}(s)$?

Update Rule for Linear Value Approximation

- The objective function is quadratic in parameters $\mathbf{w}$

$$J(\mathbf{w}) = \mathbb{E}_{\pi}[(V^{\pi}(s) - \mathbf{x}(s)^T \mathbf{w})^2]$$

- The update rule is particularly simple for a single sample

$$\begin{aligned}\Delta \mathbf{w} &= \alpha \left(V^{\pi}(s) - \hat{V}(s, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s, \mathbf{w}) \\ &= \alpha \left(V^{\pi}(s) - \hat{V}(s, \mathbf{w}) \right) \mathbf{x}(s)\end{aligned}$$

- Update = *step-size* $\times$ *prediction error* $\times$ *feature value*

Incremental *Prediction* Algorithm

- In the prior slides, we assumed access to the true value function $V^\pi(s)$.
- But in RL, we don't have such a supervision signal, but only rewards.
- So in practice, we substitute a *target* for $V^\pi(s)$:

- For MC, the target is the sample return G_t

$$\Delta \mathbf{w} = \alpha \left(G_t - \hat{V}(s, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s, \mathbf{w})$$

- For $TD(0)$, the target is the TD target $r_{t+1} + \gamma \hat{V}(s_{t+1}, \mathbf{w})$

$$\Delta \mathbf{w} = \alpha \left(r_{t+1} + \gamma \hat{V}(s_{t+1}, \mathbf{w}) - \hat{V}(s, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s, \mathbf{w})$$

- For $TD(\lambda)$, the target is the λ -return G_t^λ

$$\Delta \mathbf{w} = \alpha \left(G_t^\lambda - \hat{V}(s, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s, \mathbf{w})$$

Monte-Carlo with Value Function Approximation

- Return G_t is an *unbiased, noisy sample* of true value $V^\pi(s)$
- Can therefore apply supervised learning to “training dataset”:

$$\langle s_1, G_1 \rangle, \langle s_2, G_2 \rangle, \dots, \langle s_T, G_T \rangle$$

- For example, using *linear MC policy evaluation*

$$\begin{aligned}\Delta \mathbf{w} &= \alpha \left(G_t - \hat{V}(s, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s, \mathbf{w}) \\ &= \alpha \left(G_t - \hat{V}(s, \mathbf{w}) \right) \mathbf{x}(s)\end{aligned}$$

- This incremental approach converges to a *global* optimum
 - of Mean Squared Value Error (more on this later).
- However, it converges to a local optimum in the non-linear case.

TD Learning with Value Function Approximation

- The TD-target $r_{t+1} + \gamma \hat{V}(s_{t+1}, \mathbf{w})$ is a *biased* sample of true value $V_\pi(s)$

- We can still apply supervised learning to the "training dataset":

$$\langle s_1, r_2 + \gamma \hat{V}(s_2, \mathbf{w}) \rangle, \langle s_2, r_3 + \gamma \hat{V}(s_3, \mathbf{w}) \rangle, \dots, \langle s_{T-1}, r_T \rangle$$

- For example, using *linear TD(0)*

$$\Delta \mathbf{w} = \alpha \left(r + \gamma \hat{V}(s', \mathbf{w}) - \hat{V}(s, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s, \mathbf{w}) = \alpha \delta \mathbf{x}(s)$$

- Possible to prove that Linear TD(0) converges *close* to a global optimum
 - Specifically, converged to the *TD Fixed Point*

$TD(\lambda)$ with Value Function Approximation

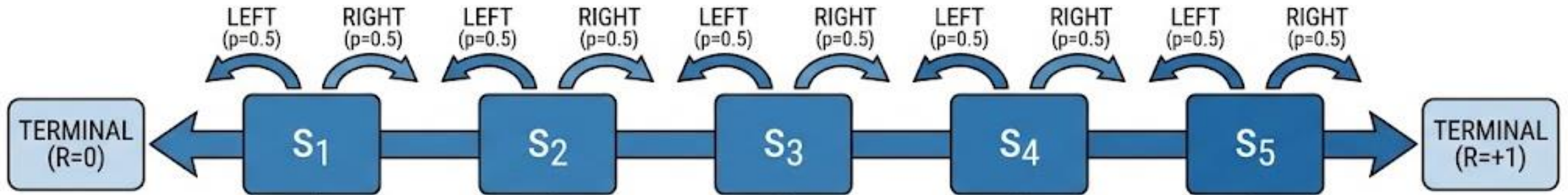
- The λ -return G_t^λ is also a *biased* sample of true value $V_\pi(s)$
- We can still apply supervised learning to the "training dataset":
 $\langle s_1, G_1^\lambda \rangle, \langle s_2, G_2^\lambda \rangle, \dots, \langle s_T, G_T^\lambda \rangle$

- *linear* $TD(\lambda)$

$$\Delta \mathbf{w} = \alpha \left(G_t^\lambda - \hat{V}(s, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s, \mathbf{w}) = \alpha \left(G_t^\lambda - \hat{V}(s, \mathbf{w}) \right) \mathbf{x}(s)$$

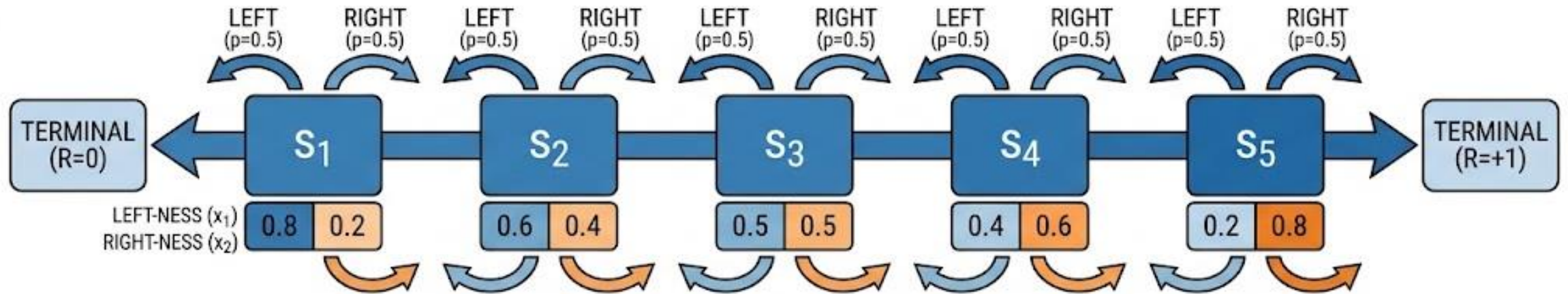
- We can use the same implementation tricks presented in prior classes to compute G_t^λ effectively

Example: Moving On a Line



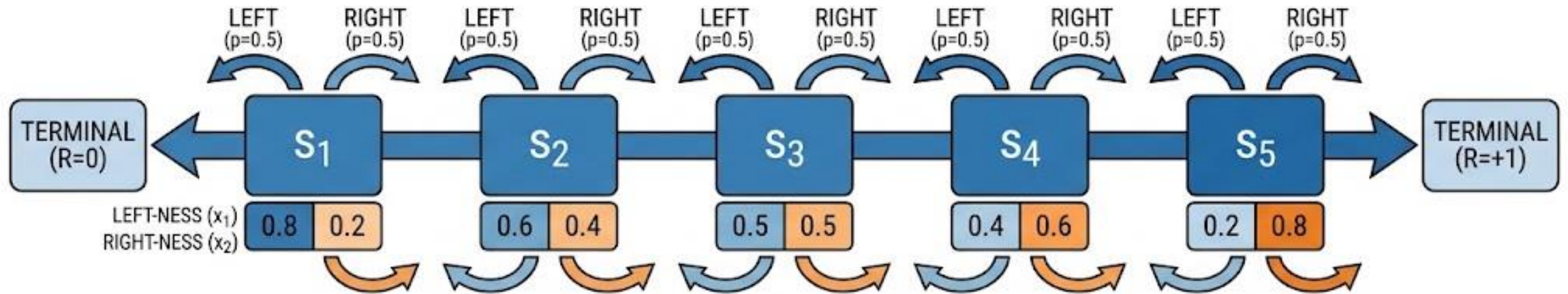
- 5 states
- π selects left or right at random
- Goal: Model-free value prediction (Find V^π given π)

Example: Moving On a Line



- Represent each state with a feature vector $x(s) = [x_1(s), x_2(s)]^T$
- x_1 is the degree of "leftness"
- x_2 is the degree of "rightness"

Example: Moving On a Line

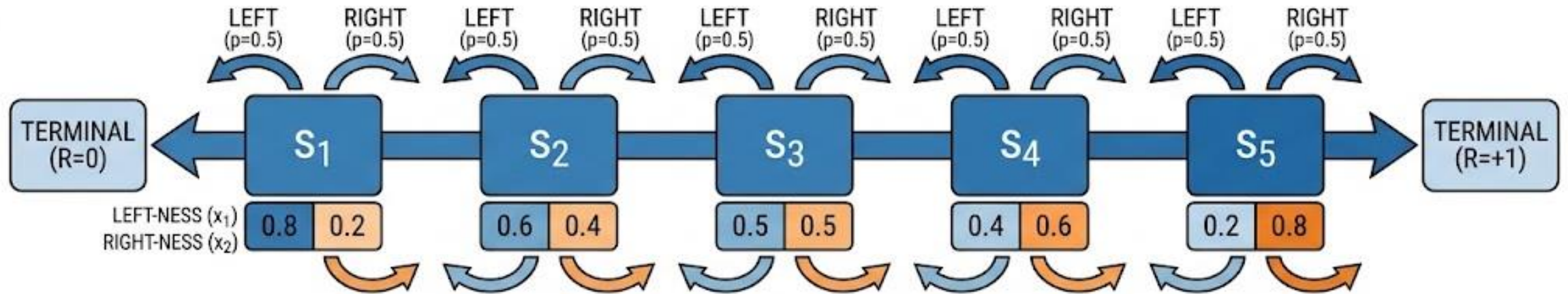


- Represent the value function as a linear function of the state:

$$\hat{V}(s, \mathbf{w}) = \mathbf{w}^T \mathbf{x}(s)$$

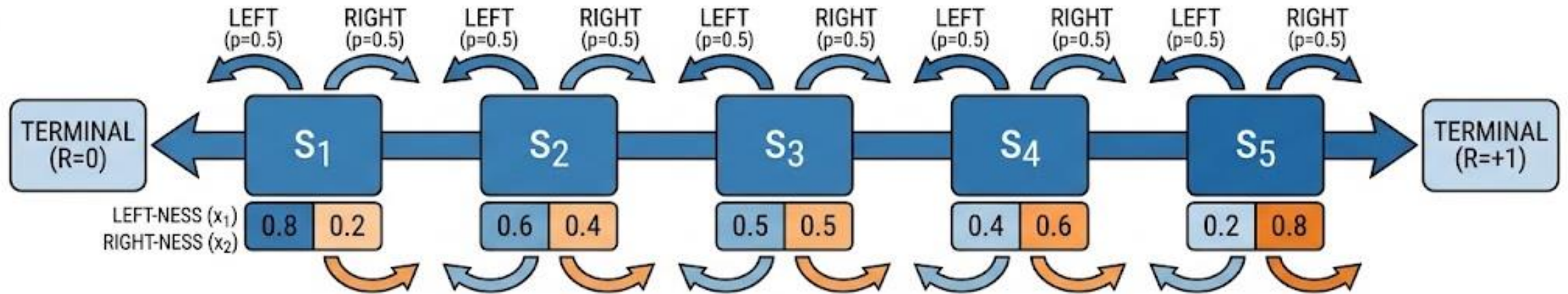
- At the beginning, $\mathbf{w} = 0$.
- So, we start with $\hat{V}(s, \mathbf{w}) = 0, \forall s$

Example: Moving On a Line



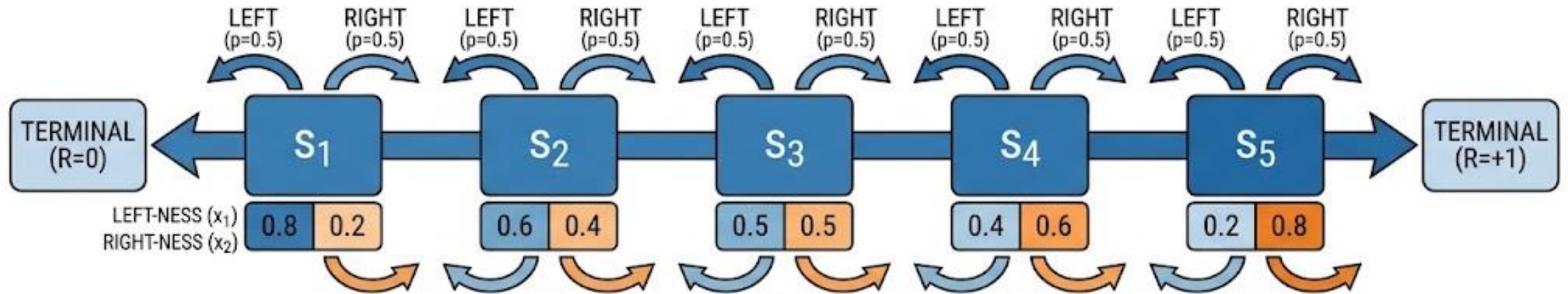
- We observe one transition: $(s_4, right, 0, s_5)$
- Compute the weight update from this transition using linear TD.
- $\Delta \mathbf{w} = \alpha \left(r + \gamma \hat{V}(s_5, \mathbf{w}) - \hat{V}(s_4, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s, \mathbf{w}) = \alpha \delta \mathbf{x}(s) = 0$
- No update!

Example: Moving On a Line



- We observe another transition: $(s_5, right, 1, T)$
- $\Delta \mathbf{w} = \alpha \left(r + \gamma \hat{V}(T, \mathbf{w}) - \hat{V}(s_5, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s, \mathbf{w}) = \alpha \delta \mathbf{x}(s)$
- $\Delta \mathbf{w} = 0.5 * 1 * [0.2, 0.8] = [0.1, 0.4]$
- $\mathbf{w}_{new} = \mathbf{w}_{old} + \Delta \mathbf{w} = [0.1, 0.4]$

Example: Moving On a Line



- Even if we have seen no transitions from prior states, we can still compute their values using the current weight:
- $\hat{V}(s_1, w) = 0.8 * 0.1 + 0.2 * 0.4 = 0.16$
- $\hat{V}(s_2, w) = 0.6 * 0.1 + 0.4 * 0.4 = 0.22$
- ...

Q Function Approximation

- Approximate the Q function: $\hat{Q}(s, a, \mathbf{w}) \approx Q^\pi(s, a)$
- Minimize the mean-squared error between approximate $\hat{Q}$ and true Q^π :

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[\left(Q^\pi(s, a) - \hat{Q}(s, a, \mathbf{w}) \right)^2 \right]$$

- Use stochastic gradient descent to find a local minimum

$$-\frac{1}{2} \nabla_{\mathbf{w}} J(\mathbf{w}) = \left(Q^\pi(s, a) - \hat{Q}(s, a, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s, a, \mathbf{w})$$

$$\Delta \mathbf{w} = \alpha \left(Q^\pi(s, a) - \hat{Q}(s, a, \mathbf{w}) \right) \Delta_{\mathbf{w}} \hat{Q}(s, a, \mathbf{w})$$

Linear Q Function Approximation

- Represent state *and* action by a *feature vector*

$$\mathbf{x}(s, a) = \begin{pmatrix} x_1(s, a) \\ \vdots \\ x_n(s, a) \end{pmatrix}$$

- Represent Q function by a linear combination of features

$$\hat{Q}(s, a, \mathbf{w}) = \mathbf{x}(s, a)^T \mathbf{w} = \sum_{j=1}^n x_j(s, a) w_j$$

- Stochastic gradient descent update on a single transition:

$$\nabla_{\mathbf{w}} \hat{Q}(s, a, \mathbf{w}) = \mathbf{x}(s, a)$$

$$\Delta \mathbf{w} = \alpha \left(Q^\pi(s, a) - \hat{Q}(s, a, \mathbf{w}) \right) \mathbf{x}(s, a)$$

Incremental Q Learning Algorithms

- Like for the Value function, we substitute a *target* for $Q^\pi(s, a)$.

- For *MC*, the target is the return G_t

$$\Delta w = \alpha \left(G_t - \hat{Q}(s_t, a_t, w) \right) \nabla_w \hat{Q}(s_t, a_t, w)$$

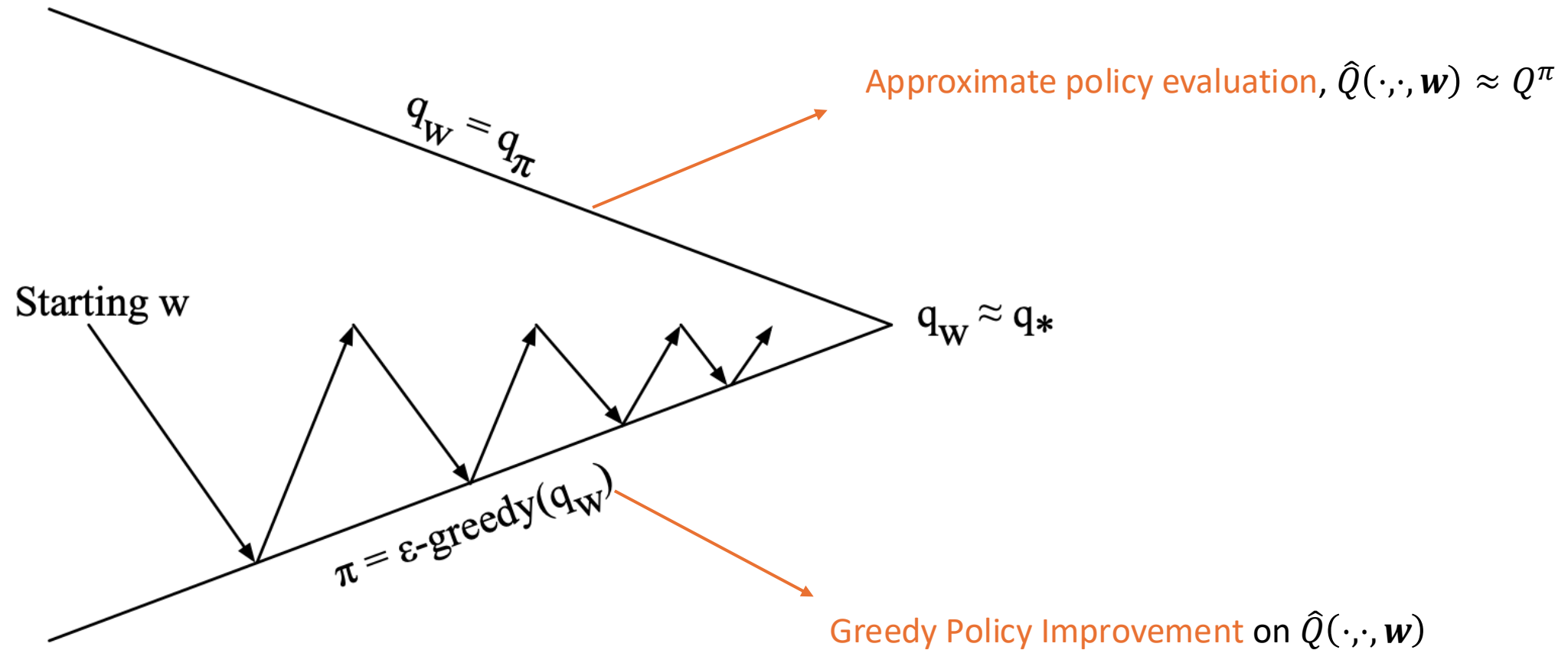
- For *TD(0)*, the target is the TD target $r_{t+1} + \gamma \hat{Q}(s_{t+1}, a_{t+1}, w)$

$$\Delta w = \alpha \left(r_{t+1} + \gamma \hat{Q}(s_{t+1}, a_{t+1}, w) - \hat{Q}(s_t, a_t, w) \right) \nabla_w \hat{Q}(s_t, a_t, w)$$

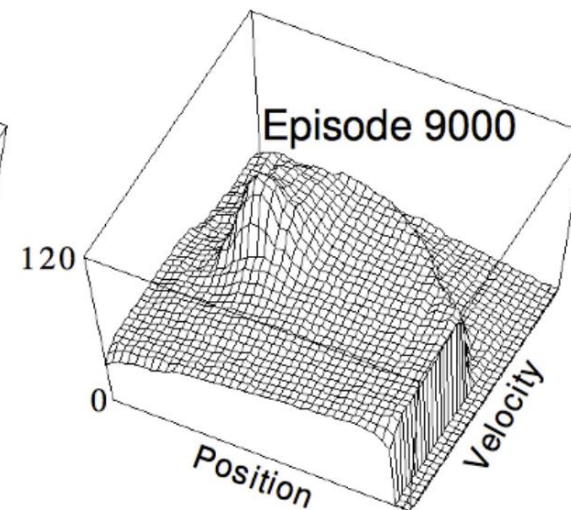
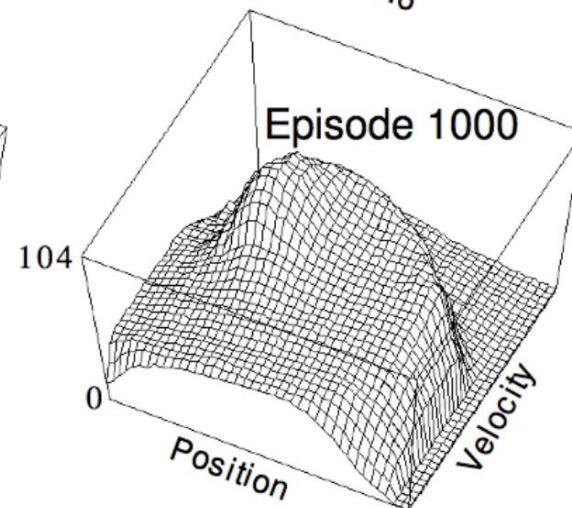
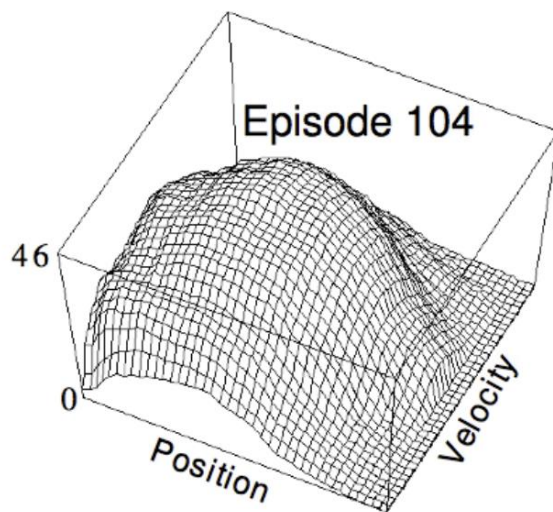
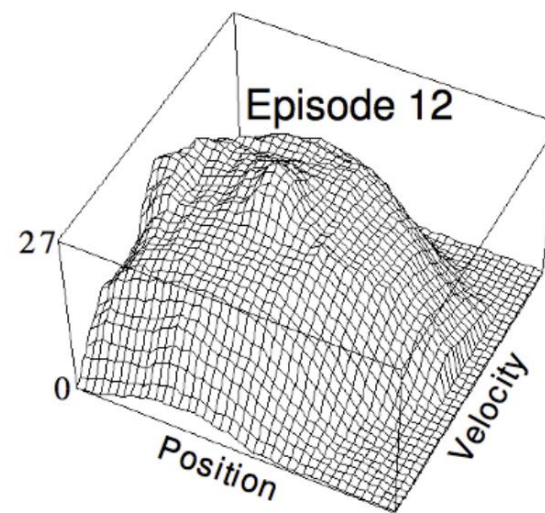
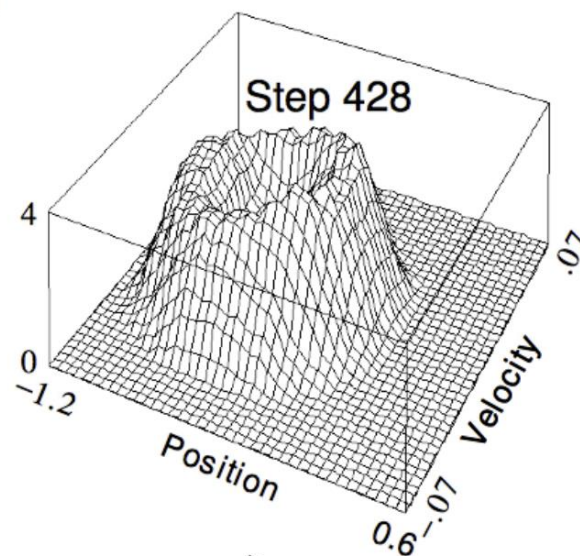
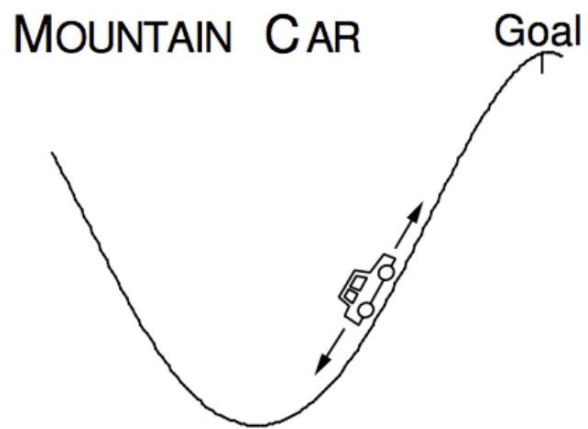
- For *TD(λ)*, the target is the action-value λ -return

$$\Delta w = \alpha \left(Q_t^\lambda - \hat{Q}(s_t, a_t, w) \right) \nabla_w \hat{Q}(s_t, a_t, w)$$

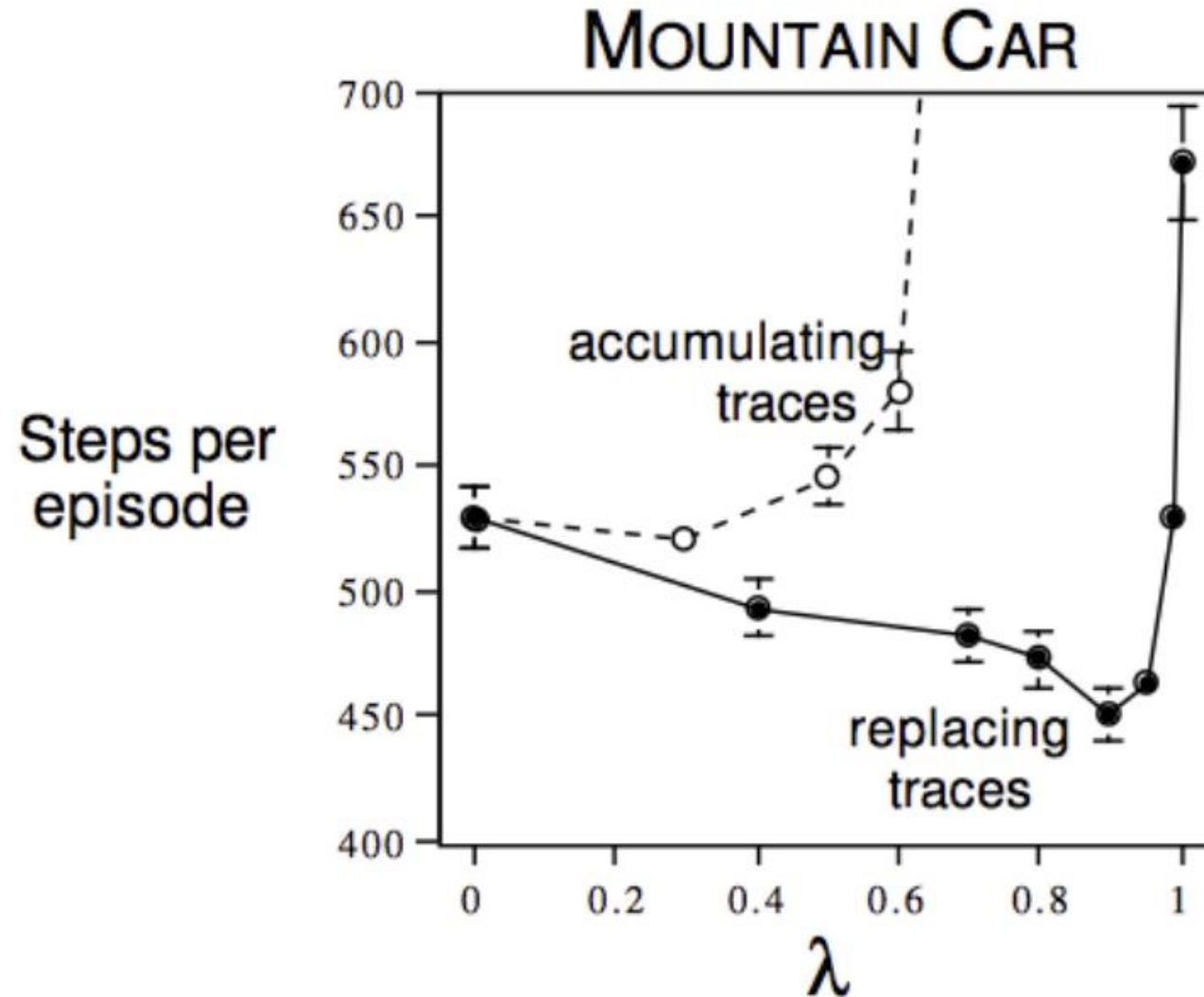
Model-free Control with Q Function Approximation



Linear Sarsa in Mountain Car



Study of λ : Should We Bootstrap?



Learning Methodologies

- *Incremental*
 - We use samples online to update the value function
 - And discard them after
- *Batch*
 - We collect many samples and then use them together to update the value fn.
 - We keep a replay buffer of old samples.

Batch Reinforcement Learning

- Online Gradient descent is simple and appealing
- But it is high variance and requires a lot of samples to converge (if it does)
- *Batch* methods seek to find the best-fitting value function
- Given a *dataset* of an agent's prior experience ("training data")

Least Square Prediction

- Given value function approximation $\hat{V}(s, w) \approx V^\pi(s)$
- And *experience* $\mathcal{D}$ consisting of $\langle \text{state}, \text{value} \rangle$ pairs
$$\mathcal{D} = \{\langle s_1, v_1^\pi \rangle, \langle s_2, v_2^\pi \rangle, \dots, \langle s_T, v_T^\pi \rangle\}$$
- Which parameters w give the *best fitting* value fn $\hat{V}(s, w)$?
- **Least squares** algorithms find a parameter vector w minimizing the sum-squared error between $\hat{V}(s, w)$ and target values V_t^π .

$$LS(w) = \sum_{t=1}^T \left(V_t^\pi - \hat{V}(s_t, w) \right)^2 = \mathbb{E}_D \left[\left(V^\pi - \hat{V}(s, w) \right)^2 \right]$$

Stochastic Gradient Descent with Experience Replay

- Given *experience* $\mathcal{D}$ consisting of $\langle state, value \rangle$ pairs

$$\mathcal{D} = \{\langle s_1, v_1^\pi \rangle, \langle s_2, v_2^\pi \rangle, \dots, \langle s_T, v_T^\pi \rangle\}$$

- **Algorithm.**

- Sample pair(s) from dataset:

$$\langle s_k, v_k^\pi \rangle \sim \mathcal{D}$$

- Apply stochastic gradient descent update

$$\Delta \mathbf{w} = \alpha \left(V^\pi - \hat{V}(s, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s, \mathbf{w})$$



Repeat

- In the linear case, it converges to the least squares solution

$$\mathbf{w}^\pi = \arg \min_{\mathbf{w}} LS(\mathbf{w})$$

Linear Least Squares Prediction

- Experience replay finds a least squares solution
- But it may take many iterations
- Using linear value function approximation $\hat{V}(s, \mathbf{w}) = \mathbf{x}(s)^T \mathbf{w}$
- We can solve the least squares problem directly.
- (not used in practice, but the derivation is fun 😊)

Linear Least Squares Prediction

- At the minimum of $LS(\mathbf{w})$, the expected update must be zero

$$\mathbb{E}_{\mathcal{D}} [\Delta \mathbf{w}] = 0$$

$$\alpha \sum_{t=1}^T \mathbf{x}(s_t)(v_t^{\pi} - \mathbf{x}(s_t)^{\top} \mathbf{w}) = 0$$

$$\sum_{t=1}^T \mathbf{x}(s_t)v_t^{\pi} = \sum_{t=1}^T \mathbf{x}(s_t)\mathbf{x}(s_t)^{\top} \mathbf{w}$$

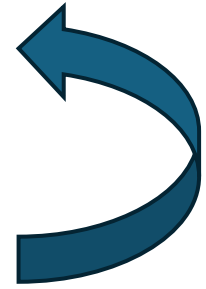
$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(s_t)\mathbf{x}(s_t)^{\top} \right)^{-1} \sum_{t=1}^T \mathbf{x}(s_t)v_t^{\pi}$$

- Computationally expensive due to matrix inversion
 - For N features, direct solution time is $O(N^3)$
 - Incremental solution time is $O(N^2)$ using Sherman-Morrison

Batch Prediction Algorithms Targets

- As for incremental methods, we do not know V_t^π
- Therefore, as “training data”, we use noisy or biased samples of it
 - MC: $V_t^\pi \approx G_t$
 - TD(0): $V_t^\pi \approx r_{t+1} + \gamma \hat{V}(s_{t+1}, \mathbf{w})$
 - $TD(\lambda)$: $V_t^\pi \approx G_t^\lambda$

Non-Linear Fn. Approx. and TD

- Assume the parametrization of $\hat{V}(s, w)$ is non-linear,
 - As, for example, when using a neural network: $\hat{V}(s, w) = NN(s; \mathbf{w})$
 - **Algorithm.**
 - Sample pair(s) from dataset:
$$\langle s_t, r_t + \gamma \hat{V}(s_{t+1}, w) \rangle \sim \mathcal{D}$$
 - Apply stochastic gradient descent update
$$\Delta \mathbf{w} = \alpha \left(r_t + \gamma \hat{V}(s_{t+1}, \mathbf{w}) - \hat{V}(s, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s, \mathbf{w}) = \alpha \delta_t(\mathbf{w}) \nabla_{\mathbf{w}} \hat{V}(s, \mathbf{w})$$
- 
- Repeat**

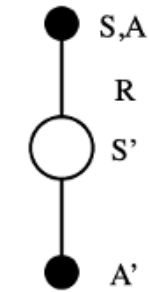
Off-Policy Learning with Function Approximation

- In the prior slides, we assumed to have a dataset of states and targets.

$$\mathcal{D} = \{\langle s_1, v_1^\pi \rangle, \langle s_2, v_2^\pi \rangle, \dots, \langle s_T, v_T^\pi \rangle\}$$

- How to get this dataset?
 - (On-policy) Collect many samples under the current policy, then learn.
 - (Off-policy) Use the data from prior policies with **off-policy targets**

Off-Policy Batch Sarsa



Sarsa

- Sample generated by an old policy (replay buffer):

$$(s_t, a_t, r_{t+1}, s_{t+1}) \sim \pi_{old}$$

- Compute target according to current policy's successor action $a' = \pi_{new}(\cdot | s_{t+1})$

$$\Delta w_k = \alpha \left(r_{t+1} + \gamma \hat{Q}(s_{t+1}, a', \mathbf{w}) - \hat{Q}(s_t, a_t, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s_t, a_t, \mathbf{w})$$

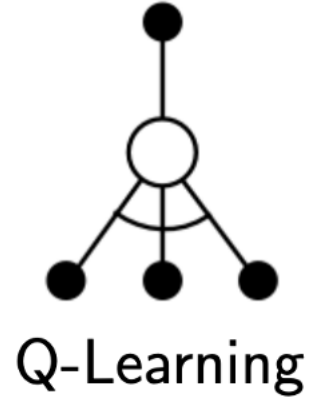
- Average update over a batch of K samples

$$\Delta \mathbf{w} = \frac{1}{K} \sum_{k=1}^K \Delta \mathbf{w}_k$$
$$\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w}$$

- Interact with the environment using $\pi_{new} = \epsilon\text{-greedy}(\hat{Q})$, adding experiences to the replay buffer.

Repeat

Off-Policy Batch Q-Learning



- Sample generated by an old policy (replay buffer):

$$(s_t, a_t, r_{t+1}, s_{t+1}) \sim \pi_{old}$$

- Compute target according to Bellman Optimality Equation:

$$\Delta w = \alpha \left(r_{t+1} + \gamma \max_{a_{t+1}} \hat{Q}(s_{t+1}, a_{t+1}, w) - \hat{Q}(s_t, a_t, w) \right) \nabla_w \hat{Q}(s_t, a_t, w)$$

- Average update over a batch of K samples

$$\Delta w = \frac{1}{K} \sum_{k=1}^K \Delta w_k$$

$$\hat{Q}(s, a, w) \leftarrow \hat{Q}(s, a, w + \Delta w)$$

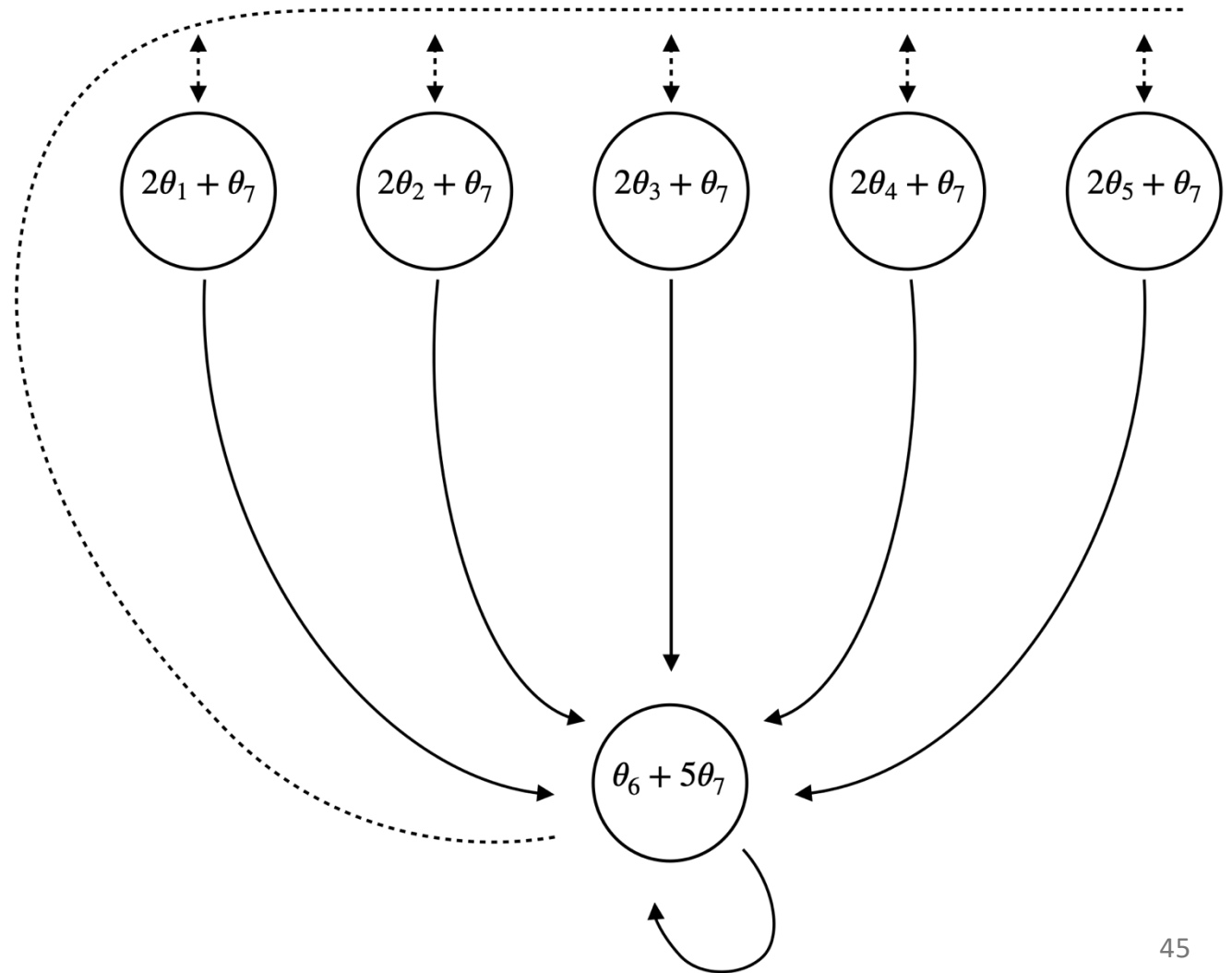
- Interact with the environment using $\pi_{new} = \epsilon\text{-greedy}(\hat{Q})$, adding experiences to the replay buffer.

Repeat

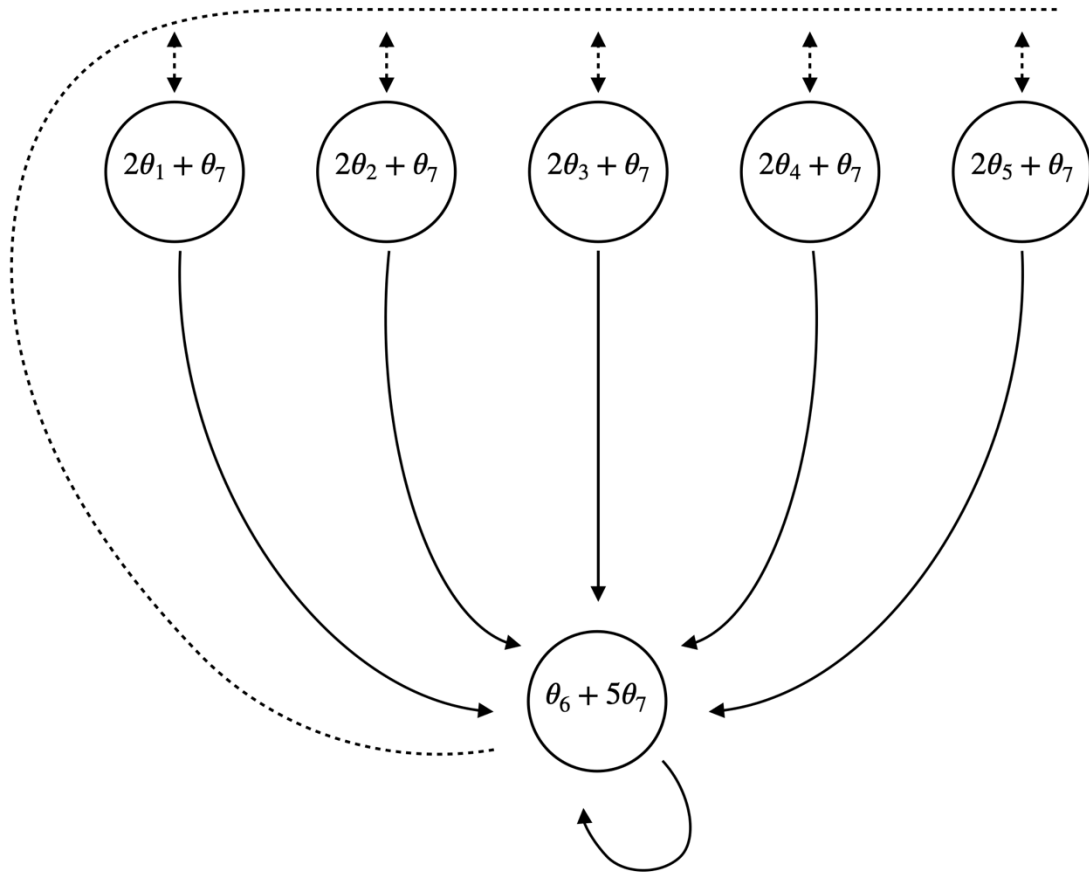
Baird's Deadly Triad (1995)

Risk of divergence when combining:

- Off-policy training,
- Function approximation,
- Bootstrapping.

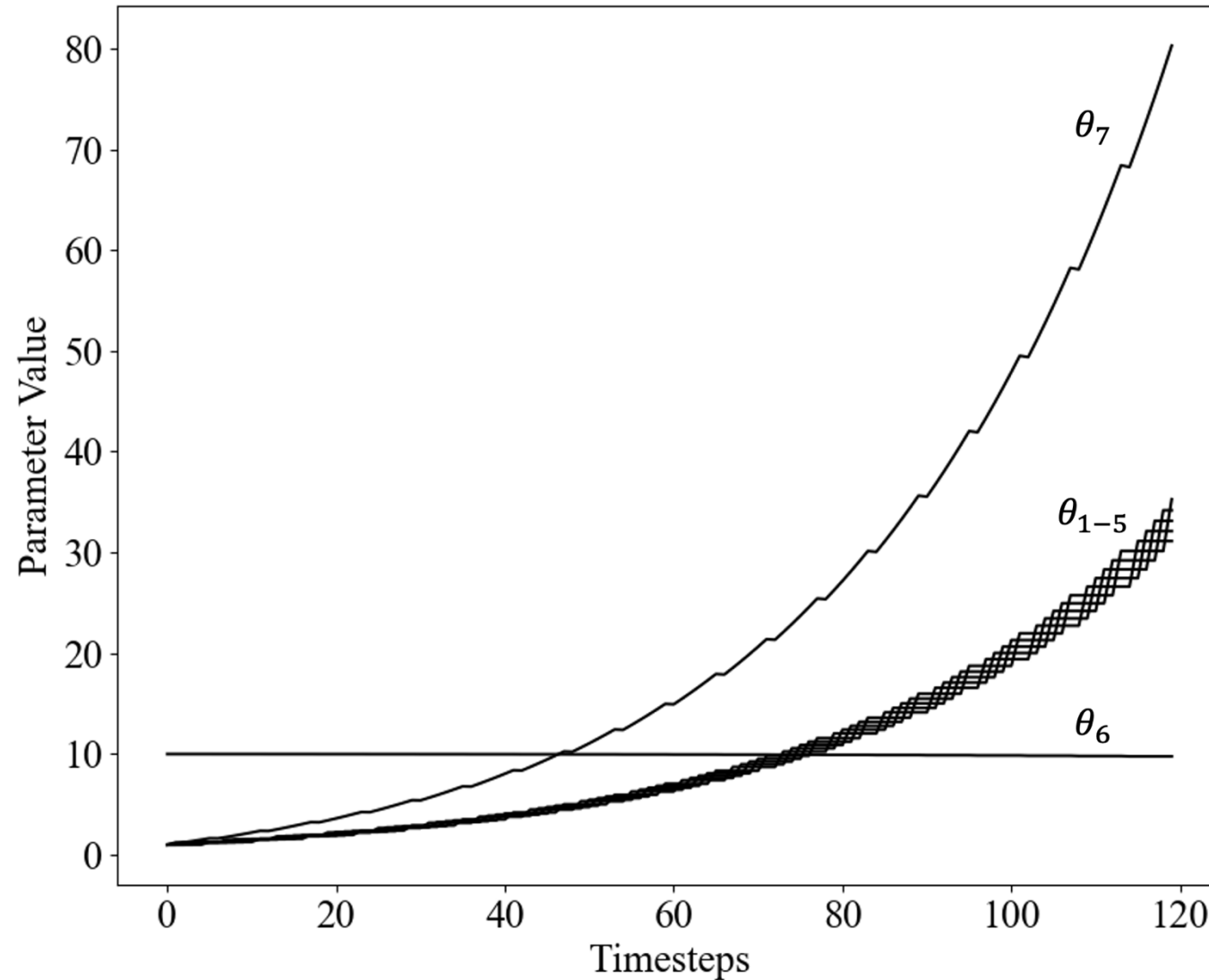


Baird's Deadly Triad (1995)



- $r_t = 0$
- $V^\pi(s) = 0, \forall \pi$
- $w = [\theta_1, \dots, \theta_7]$
- Numbers in nodes represent $\hat{V}(s, w)$
- $\mu(\text{dashed}) = \frac{1}{6}, \mu(\text{solid}) = \frac{5}{6}$
- $\pi(\text{solid}) = 1$
- The lower state is terminal

Baird's Deadly Triad (1995)



Source: [this link](#)

Convergence of Prediction Algorithms

On/Off-Policy	Algorithm	Table Lookup	Linear	Non-Linear
On-Policy	MC	✓	✓	✓
	TD(0)	✓	✓	✗
	TD(λ)	✓	✓	✗
Off-Policy	MC	✓	✓	✓
	TD(0)	✓	✗	✗
	TD(λ)	✓	✗	✗

Convergence of Prediction Algorithms

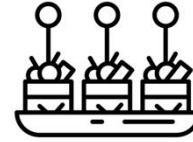
- We cannot prove the convergence of TD algorithms for non-linear functions 😞
- A variant of TD learning ([Gradient TD](#)), which follows the true gradient of the projected Bellman error, also converges in the linear and non-linear case.
 - But not used in practice (particularly in RL with neural nets).
- We use other ideas instead
 - More on in the next classes

Convergence of *Control* Algorithms

Algorithm	Table Lookup	Linear	Non-Linear
Monte-Carlo Control	✓	(✓)	✗
Sarsa	✓	(✓)	✗
Q-learning	✓	✗	✗
Gradient Q-learning	✓	✓	✗

(✓) = chatters around near-optimal value function

Deep Q-Networks (DQN)



- $\Delta w = \alpha \left(r_{t+1} + \gamma \max_{a_{t+1}} \hat{Q}(s_{t+1}, a_{t+1}, \mathbf{w}) - \hat{Q}(s_t, a_t, \mathbf{w}) \right) \nabla_w \hat{Q}(s_t, a_t, \mathbf{w})$
- **Problem:** Maximization bias.
- **(Potential) Solution (DQN, Mnih et. al 2013):**
 - Use the parameters from the prior optimization step, w_{i-1} , in the maximization step.
 - $\Delta w = \alpha \left(r_{t+1} + \gamma \max_{a_{t+1}} \hat{Q}(s_{t+1}, a_{t+1}, \mathbf{w}_{i-1}) - \hat{Q}(s_t, a_t, \mathbf{w}_i) \right) \nabla_w \hat{Q}(s_t, a_t, \mathbf{w}_i)$
- We'll speak of other algorithms and solutions later in the class.